41052 Advanced Algorithms
Programming Assignment 2
Spring 2026

Deterministic vs Randomised
String Matching

Karp-Rabin, KMP, Boyer-Moore
Compared

Table of Contents

Symbols and terms used 4

1 Problem statement 5

1.1 Karp-Rabin 5

1.2 Knuth-Morris-Pratt (KMP) 6

1.3 Boyer-Moore 7

1.4 The baseline: std::string::find 8

2 Theoretical comparison and predictions 9

2.1 Comparison on paper 9

2.2 Predictions 10

3 Experimental methodology 11

3.1 Input families and why we chose them 11

3.2 Fairness and Reproducibility 12

3.3 Correctness testing 12

3.4 Karp-Rabin collision experiments 13

4 Results and analysis 13

4.1 Pattern length 13

4.2 Alphabet size 16

4.3 Text length 17

4.4 Repetitive and worst-case inputs 17

4.5 Karp-Rabin Random Base 19

4.6 Karp-Rabin Tail Behaviour 20

5 What surprised us 21

5.1 Binary text: Boyer-Moore beat KMP 21

5.2 Karp-Rabin collisions were free for short patterns 21

5.3 KMP slowed down on small alphabets 22

5.4 Longer text costs more per character on small alphabet size 23

6 What we'd do next 24

7 Individual contributions 24

8 AI use 25

8.1 How each member used AI 25

8.1.1 Aeesha Bello 25

8.1.2 Mallory Li 25

8.1.3 Rebecca Kim 25

8.2 Where AI was wrong, unhelpful, or misleading 26

8.3 AI in group work 27
9 References 28
10 Appendix A: Individual reflections 29
10.1 Karp-Rabin – Mallory Li 29
10.2 KMP – Aeesha Bello 29
10.3 Boyer-Moore – Rebecca Kim 30

Symbols and terms used

Symbol or term	Meaning	Example
Text	The long string that is searched within	AHBSNSMSKGATTACAJKSO
Pattern	The short string that we are looking for	GATTACA
n	Length of the text, in characters	$n = 2^{20} \approx 1$ million
m	Length of the pattern, in characters	GATTACA has m = 7
σ (sigma)	alphabet size: how many different characters can appear	4 for DNA (A, C, G, T), 26 for lowercase letters, 256 for any byte
Match	A place where the pattern appears in the text	
Position	Where a match starts, counting from zero	in banana, **ana** matches at positions 1 and 3
a^n, $a^{m-1}b$	shorthand for repeated letters: a^n is the letter **a** written n times; $a^{m-1}b$ is m − 1 a's followed by one b	a^4 = aaaa; for m = 4, $a^{m-1}b$ = aaab
2^{16}, 2^{20} , 2^{24}	powers of two, used for sizes	2^{16} = 65,536; $2^{20} \approx 1$ million; $2^{24} \approx 16.8$ million
Shift	How far the pattern moves along the text after a check	
Window	The text we are looking through. This is the m characters of text that start at one position	
Alignment	Placing the pattern under one window to check it	

How fast an algorithm grows

We describe cost with **O** notation. This says how the running time grows as the input grows.

- O(n), linear: doubling the text doubles the time. One pass over the text.
- O(n/m), sublinear: the algorithm skips most of the text, so longer patterns make it faster.
- O(nm): time grows with text length times pattern length. With n = 1 million and m = 1,000, that is about a billion steps instead of a million
- Worst case: the slowest any input can make the algorithm. **Expected**: the average over the algorithm's own random choices, for a given input.

1 Problem statement

Our chosen problem was how input characteristics affect the performance of deterministic and randomised string-matching algorithms. Given a text of length n and a pattern of length m, find every position where the pattern appears, including matches that overlap. For example, in the text **aaaa** the pattern aa appears at positions 0, 1 and 2. If the pattern is empty or longer than the text, there are no matches.

We compared three algorithms, Knuth-Morris-Pratt, Boyer–Moore and Karp–Rabin across different text lengths, pattern lengths, alphabet sizes, and input structures. We also built inputs that make Karp-Rabin's hashes collide, to measure the cost of its false positive checks. We time std::string::find from the C++ standard library as a baseline (section 1.4). All our implementations take the same two inputs and return the same output: the list of match positions in increasing order.

The simple method ("naive search") places the pattern at every position and compares character by character until a mismatch. That needs up to m comparisons at each of about n positions: Θ(nm) in the worst case. Each algorithm below avoids that waste in a different way.

1.1 Karp-Rabin

Karp-Rabin is the randomised algorithm in our comparison, and instead of comparing characters straight away it turns the pattern into a single number called a hash and then works out the same kind of number for every window of m characters in the text. Only the windows whose hash equals the pattern's hash are compared character by character, so most windows are ruled out with one comparison between two numbers.

The rolling hash

The hash treats a window as a number in some base, kept modulo a large prime (the modulus). For abc it is (a × base + b) × base + c, using byte values. The next window's hash comes from the current one in constant time: take away the first character times base to the power m – 1, multiply by the base and add the new character. My rabin_karp.cpp does this with two multiplications and three modulo operations per text character, whatever the pattern length is.

Why the base is random

Two different windows can have the same hash, which is called a collision, so every hash hit is checked against the text, before it is reported. A hit that fails this check is a false positive. This means Karp-Rabin never returns a wrong match, and the randomness only affects its running time.

My implementation uses the prime 2,147,483,647 ($2^{31} - 1$) as the modulus and picks a new random base on every call. The hashes of two different windows differ by a polynomial in the base of degree less than m, and modulo a prime such a polynomial has at most m − 1 roots. So for any fixed text and pattern, each window gives a false positive with probability at most (m − 1)/(modulus − 2), about 1 in 140 million for m = 16. This probability is over the algorithm's choice of base, not over the text, so it holds for every input, even one built to cause collisions, as long as whoever built it cannot see the base.

Worked example
Searching for ana in banana with base 10 and a = 1, b = 2, n = 3, the pattern's hash is 131. The windows ban, ana, nan and ana hash to 213, 131, 313 and 131, so only positions 1 and 3 are checked.

Complexity
Preprocessing takes O(m) time and a few numbers of memory. With a random base the expected search time is O(n + m) plus m for each real match. The worst case is Θ(nm): with a^m in a^n every window is a real match and is checked in full, and no base can avoid this because these hits are not collisions.

How it differs from KMP and Boyer-Moore
Karp-Rabin uses arithmetic instead of comparisons and tables, so its cost per character does not depend on the alphabet or the pattern length. It never skips text like Boyer-Moore, has no linear worst case like KMP, and is the only one of the three whose time on the same input can change between runs.

1.2 Knuth-Morris-Pratt (KMP)

The Knuth-Morris-Pratt algorithm is a string-searching algorithm used to find all occurrences of a pattern within a larger text. Unlike the naive string matching algorithm, KMP avoids re-checking characters that have already been matched, making it more efficient for large inputs.

The main idea behind KMP is that when a mismatch occurs, the algorithm already knows some information about the pattern from previous matches. Instead of moving back in the text and starting the comparison again, KMP uses this information to determine how far the pattern can be shifted. To do this, KMP preprocesses the pattern and creates a table called the Longest Prefix Suffix (LPS) array. The LPS array records the length of the longest proper prefix that is also a suffix for each position in the pattern.

For example, for the pattern: **ABAB**

the LPS array is: **0 0 1 2**

This means that when a mismatch occurs after matching "ABAB", the algorithm knows that "AB" has already been matched and does not need to be compared again.

How Does It Work?

KMP has 2 stages. The first is preprocessing the pattern. The algorithm scans the pattern once and builds the LPS array. This table determines how much of a previous match remains useful after a mismatch. The second is searching text. The algorithm compares the pattern against the text character by character.

- If characters match, both indices move forward.
- If a mismatch occurs, the text index stays where it is.
- The pattern index is moved according to the LPS table.
- Previously matched characters are not rechecked.

Complexity

Operation	Complexity
Building the LPS table	O(m)
Searching the text	O(n)

Therefore, the complexity of the algorithm is **O**(n+m). The space complexity of the algorithm is simply O(m), as space is required to store the LPS array for the pattern.

What makes KMP Different?

When a mismatch occurs, KMP uses the LPS values to determine how much of the previous match is still valid. This allows it to continue searching without re-examining characters that are already known to match. Because of this optimisation, KMP guarantees linear-time searching even in worst-case inputs. Boyer-Moore and Karp-Rabin both slow to Θ(nm) in their worst case.

1.3 Boyer-Moore

Boyer-Moore lines the pattern up under the text and compares it from right to left, starting with the pattern's last character (Boyer & Moore, 1977). Before the search, it builds two tables from the pattern. When a character doesn't match, these tables say how far the pattern can safely move right. It moves by the larger of the two.

The two shift rules

- Bad character rule

  Look at the text character that caused the mismatch. Move the pattern so this character lines up with its last copy in the pattern. If the pattern doesn't contain it, move the pattern right past it. If its last copy is to the right of the mismatch, this rule gives no useful shift and the good suffix rule decides.
- Good suffix rule

  Look at the characters that did match, which form a suffix of the pattern. Move the pattern to the next copy of that suffix that has a different character in front of it. If there is no such copy, line up the longest suffix of the matched part that is also a prefix of the pattern (GeeksforGeeks, 2025). src/boyer_moore.cpp uses this strong form of the rule, because a copy with the same character in front would fail at the same place again.

After a full match, the pattern moves by goodSuffix[0], not by m. This is the pattern's period, the smallest shift that lines the pattern up with itself again. It equals m minus the length of its longest border (a part at the start of the pattern that also appears at the end). This way overlapping matches such as aa in aaaa are still found.

Worked example

Searching for EXAMPLE in HERE IS A SIMPLE EXAMPLE (one of the tests in tests/boyer_moore_tests.cpp):

- The first alignment compares the pattern's last letter E with the text's S.
- S isn't in EXAMPLE, so the bad character rule moves the pattern 7 places, past the S.
- boyer_moore_search finds the match at position 17 after 5 alignments and 15 character comparisons. A naive search would try all 18 positions.

Complexity

Preprocessing takes O(m + 256) time and space: one bad character entry for each of the 256 byte values, and a good suffix table built in O(m) from borders. In the best case, when most text characters are not in the pattern, it needs about n/m alignments with one comparison each, so most of the text is never read. In the worst case, such as a^m in a^n, every alignment compares all m characters and then moves one place, which is Θ(nm).

How it differs from KMP and Karp-Rabin

Boyer-Moore is the only one of our three that can skip text without reading it, so its speed depends strongly on the alphabet size and the pattern length, which decide how far it can skip.

1.4 The baseline: std::string::find

We also time std::string::find from the C++ standard library, as a reference for what a well-optimised library achieves, and every algorithm's answer is checked against it. find only returns the first match after a position, so stdFindSearch (in include/test_utils.hpp) calls it again from one past each match, which also finds overlapping matches. Its worst case is Θ(nm): on $a^{m-1}b$ in a^n the first character matches everywhere, so it can compare up to m characters at every position.

2 Theoretical comparison and predictions

We wrote this section before running any timing experiments.

2.1 Comparison on paper

Table 1. The three algorithms on paper (n = text length, m = pattern length, σ = alphabet size).

	Karp-Rabin	**KMP**	**Boyer-Moore**	**std::string::find**
Type	Randomised (always correct, only time is random)	Deterministic	Deterministic	Deterministic
Preprocessing	O(m) Hash of the pattern and first window + base to the power m – 1	build failure/LPS table: O(m)	O(m +256) two tables	None
Search, best case	Θ(n)	O(n)	O(n/m)	Θ(n)
Search, worst case	Θ(nm) when almost every window is a real match or a collision	O(n), at most 2n comparisons	Θ(nm): a match at almost every position (a^m in a^n)	Θ(nm)
Expected search time	O(n + m) plus m for each real match	O(n), n to 2n comparisons	Sublinear on large alphabets	Close to Θ(n) on random text
Work per text character	One hash update, which is 2 multiplications and 3 modulo operations	1-2 character comparisons	Under 1 comparison on large alphabets, up to m on repetitive text	A memchr scan for the first character, then a check of the rest
Extra memory	O(1), a few 64-bit numbers	One number per pattern character, O(m)	O(m + 256): 256 + 2(m + 1) ints	O(1)
Most sensitive to	How many windows have the same hash as the pattern	Alphabet size: with 2 or 4 letters, more partial matches fail and fall back through the LPS table	Alphabet size and m (sets how far it skips)	How often the pattern's first character appears in the text

On ordinary text all four should take time roughly in proportion to n, so the differences should come from whether an algorithm can skip text and how much work it does per character. KMP does one or two cheap comparisons per character but reads every one. Boyer-Moore does more work per alignment, but on a large alphabet it only needs about n/m alignments, so it should win once m is more than a few characters. Karp-Rabin does several arithmetic operations per character, so it should be the slowest on ordinary text. The extra memory each algorithm needs is a few kilobytes at most (Table 1), while our texts are up to 16 MB, so we don't expect memory use to affect the times.

2.2 Predictions

1. **P1: Random text**
   On random text, Boyer-Moore will be the fastest for all but the shortest patterns tested. Its lead will grow as m grows. We expect *find* to beat all three for short patterns ($m \leq 8$), since it only needs a fast scan for the first character. We expect Boyer-Moore to overtake *find* somewhere between $m = 16$ and $m = 32$. Karp-Rabin will be the slowest.

2. **P2: Crossover in Pattern Length**
   KMP will beat Boyer-Moore for very short patterns ($m < 2$), but Boyer-Moore will be faster once the pattern reaches roughly 4 to 8 characters. Short patterns means there is little opportunity to skip text, so Boyer-Moore, which relies on preprocessing, will struggle with the overhead.

3. **P3: Alphabet Size**
   Boyer-Moore's performance should increase as the alphabet size increases. This is because a large alphabet size makes accidental character matches less likely. Boyer-Moore will then have bigger shifts with fewer comparisons. KMP and Karp-Rabin will be sensitive to the alphabet size. KMP due to the structure of the pattern itself rather than the alphabet size.

4. **P4: Text Length**
   We believe the three algorithms should have linear scaling with respect to the text length. KMP does a constant amount of work per character, and Karp-Rabin does one rolling hash update per window. Boyer-Moore can skip large parts of the text, but its alignments should still grow in proportion to the text length.

5. **P5: Repetitive text with many matches**
   KMP is fastest, and its time does not change with m. Boyer-Moore and Karp-Rabin grow linearly in m, so we expect KMP to be ahead from about $m = 4$, and more than 10 times ahead by $m = 100$.

6. **P6: Pattern aa…ab Against Text aa…aa**
   Boyer-Moore should outperform the other algorithms. The only mismatch appears at the final character. KMP will always reach the mismatch and do a fall back through the LPS table. Boyer-Moore will compare from right to left and will find that the last character does not match.

7. **P7: Deliberately colliding input (rk_adversarial)**
   With base 31, Karp-Rabin will slow down to $\Theta(nm)$. We believe the random-base version's running time will not be affected.

3 Experimental methodology

3.1 Input families and why we chose them

All the inputs come from benchmark/input_generators.cpp. Each experiment draws its inputs from its own seed range, starting from the master seed 20261029 (recorded in results/environment.txt), so every run builds exactly the same strings. Each input is built once, before any timing, and the same text and pattern go to every algorithm.

Random text picks each character uniformly from the first σ symbols of a fixed list (a-z, A-Z, 0-9, then other byte values), and the pattern is cut from the text, so there is always a match. Built text is a^n with a chosen pattern.

Table 2. The six experiments

Experiment	Text and pattern	What is varied	Why this input
scale_n	Random. Each n uses the first n characters of one long text, with the same pattern.	n from 2^{14} to 2^{24} m = 16 σ = 4 and 26	Checks that time grows in proportion to n. Gives each algorithm's cost per character.
scale_m	Random	m from 2 to 1,024 n = 2^{22} σ = 4 and 26	Boyer-Moore's skips should grow with m. KMP and Karp-Rabin should stay flat.
alphabet	Random	σ = 2, 4, 8, 16, 32, 64, 128, 256 n = 2^{22} m = 32	Small alphabets give short bad-character shifts and longer partial matches Large alphabets do the opposite.
repetitive	Text a^n, with one of these patterns: a^m (matches everywhere) $a^{m-1}b$ (fails on the last character) ba^{m-1} (fails on the first character)	m from 4 to 1,024 n = 2^{18}	These are the worst cases for Boyer-Moore and Karp-Rabin (a^m) and for find ($a^{m-1}b$).
rk_adversarial	Text a^n A pattern whose hash matches every window when the base is 31	m from 8 to 1,024 n = 2^{18}	Pushes fixed-base Karp-Rabin to Θ(nm). Compares it with the random base on the same input.
rk_tail	Each input searched 1,000 times A new random base each time	Random text, 4 letters, n = 2^{20}, m = 16, with the normal modulus ($2^{31} - 1$) Random text, 26 letters, n = 2^{16}, m = 16, with small moduli (101, 1,009 and 10,007) The adversarial input, n = 2^{16}, m = 256, with moduli 101, 1,009 and $2^{31} - 1$	How much Karp-Rabin's time and false positives depend on its base. Small moduli make false positives common enough to count.

The adversarial pattern is m – 8 a's followed by eight lowercase letters that differ from aaaaaaaa but have the same hash for base 31. The generator finds them by searching letter combinations. Every window of a^n then has the pattern's hash, and each check compares at least m – 8 characters before it fails.

Benchmark setup

We first had 11 repetitions. This run produced variation of results for a variety of experiments which led to some median values being out of order. Due to this, we increased the number of repetitions to 21 for all algorithms and reran the benchmarking. All raw measurements are in **results/runtime.csv**, and **results/summary/runtime_summary.csv** gives the median, quartiles and spread for each setting.

For every instance, **benchmark/benchmark_runner.cpp** does the following:

- Each algorithm runs once as a warm-up, which is not one of the timed samples, and its result must equal std::string::find's exactly. Because every algorithm is compared with the same answer, the exact algorithms also agree with each other, and the randomised algorithm is checked against a deterministic answer on every instance.
- In each of the 21 timed rounds, the five searches (our three, Karp-Rabin with base 31, and find) run in a freshly shuffled order, so changes in machine load during the run are spread evenly across all of them.

Once timing is complete, each algorithm runs once more in a **counting** version of the same code. This records character comparisons, Boyer-Moore alignments, KMP fallbacks, and Karp-Rabin hash hits and false positives. For KMP and Boyer-Moore these are separate functions, so counting does not slow down the timed version. Karp-Rabin's timed search calls its counting version, but its counters only change on a hash hit.

3.2 Fairness and Reproducibility

Every algorithm gets the same strings, built from the fixed seeds, and must return the same list of positions. Making the inputs, checking the answers and writing the results all happen outside the timed section, so we only time the search. Each algorithm's setup work (tables for KMP and Boyer-Moore, the pattern hash for Karp-Rabin) is timed, because it is part of the cost of using that algorithm. Each of us wrote a plain version of our algorithm as described in section 1, and all three are built into the same program with the same compiler and flags.

The whole pipeline is one command: python scripts/reproduce.py (on Windows, py scripts\reproduce.py). It builds in Release mode with CMake, runs the four test programs and stops if any fails, deletes the old results/ folder, runs the benchmark and the explanation tests, and regenerates every plot and summary table. Every number and figure in this report comes from the committed files in results/.

3.3 Correctness testing

Before any timing, ctest runs four test programs, and reproduce.py stops if any fail.

tests/cross_tests.cpp (16 checks) compares every benchmarked algorithm, including both Karp-Rabin versions, with a naive search on every small text and pattern over {a, b} and {a, b, c} (over 500,000 cases), on 25,000 random cases and on every input family. It also uses moduli 7 and 13 to force collisions and check Karp-Rabin stays exact.

tests/kmp_tests.cpp (29 checks) tests the LPS table and the 2n bound.

tests/boyer_moore_tests.cpp (29 checks) runs the same {a, b} cases on Boyer-Moore and checks the good suffix table against a slow version written from the rule's definition. tests/stringmatching_tests.cpp runs hand-worked cases such as ana in banana on all three, then compares them on 10,000 random texts over four letters.

3.4 Karp-Rabin collision experiments

Karp-Rabin is the only algorithm whose speed depends on a random choice, so we tested that choice in two ways. rk_adversarial (Table 2) shows what the fixed base costs on input built against it.

rk_tail keeps the input fixed and changes only the base. Each of its seven settings (Table 2) is searched 1,000 times with a new base, timed once per trial, and the same run counts hash hits and false positives. KMP and Boyer-Moore are timed 1,000 times on the same inputs, as a baseline without randomness. For moduli 101 and 1,009 the benchmark also tries every base to find the exact fraction that make the adversarial pattern collide. The results are in results/summary/rk_tail_summary.csv.

4 Results and analysis

All times below are medians of 21 measurements in ns/char (nanoseconds per text character and lower is better).

4.1 Pattern length

Table 3. Random text, $n = 2^{22}$, $m = 16$.

Method	ns/char, σ = 4	ns/char, σ = 26	comparisons/char, σ = 4	comparisons/char, σ = 26
KMP	3.65	1.36	1.234	1.038
Boyer–Moore	1.44	0.39	0.262	0.089
Karp–Rabin (random base)	9.07	9.10	0.000	0.000
Karp–Rabin (base 31)	9.00	8.95	0.000	0.000
std::string::find	0.25	0.11	—	—

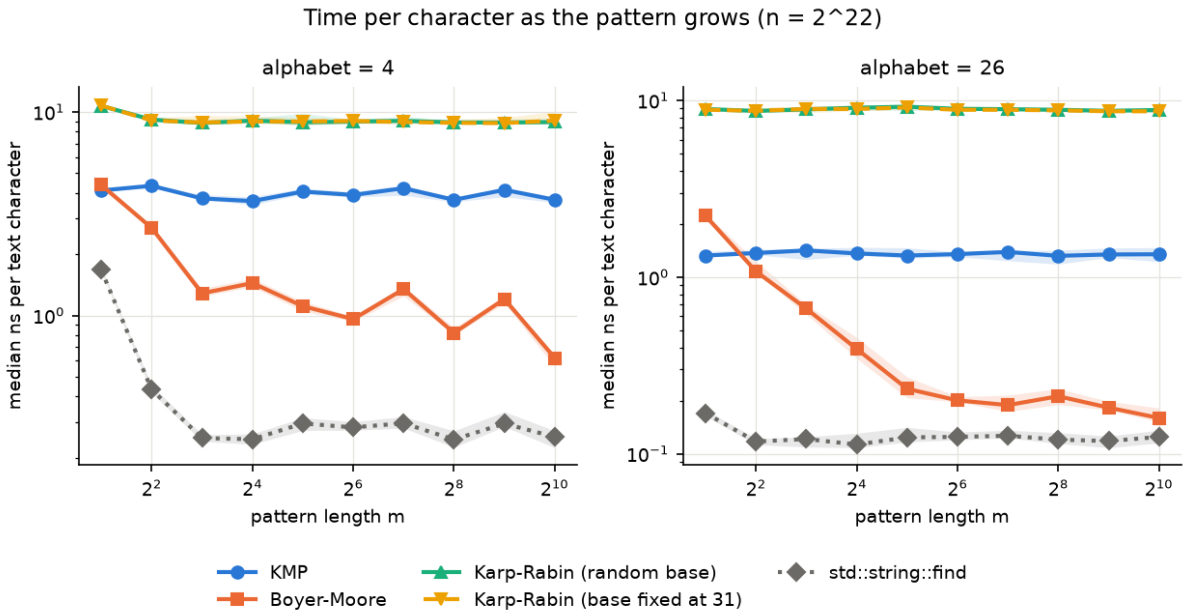


Figure 1. Time per character against pattern length on random text ($n = 2^{22}$)

Figure 1 shows that Boyer-Moore outperforms KMP for all pattern lengths m>= 4, with the advantage growing from 2.5× at σ=4 to 3.5× at σ=26. This is because Boyer-Moore's bad-character and good-suffix heuristics allow it to skip many alignments entirely, reducing its work to just 0.089 comparisons per character at m=16 and σ=26, compared with KMP's 1.038. As the alphabet grows, mismatching characters are less likely to appear elsewhere in the pattern, increasing the average shift distance and further reducing comparisons. KMP remains close to one comparison per character because it processes the text sequentially and uses the LPS table to avoid rechecking characters rather than skipping ahead. Karp-Rabin is the slowest despite performing almost no character comparisons on random text, as its runtime is dominated by rolling-hash computations performed for every text window. The most surprising result is that std::string::find is faster than all three algorithms across all tested pattern lengths, suggesting that its highly optimised library implementation and hardware-level optimisations outweigh the theoretical advantages of the specialised algorithms on this workload.

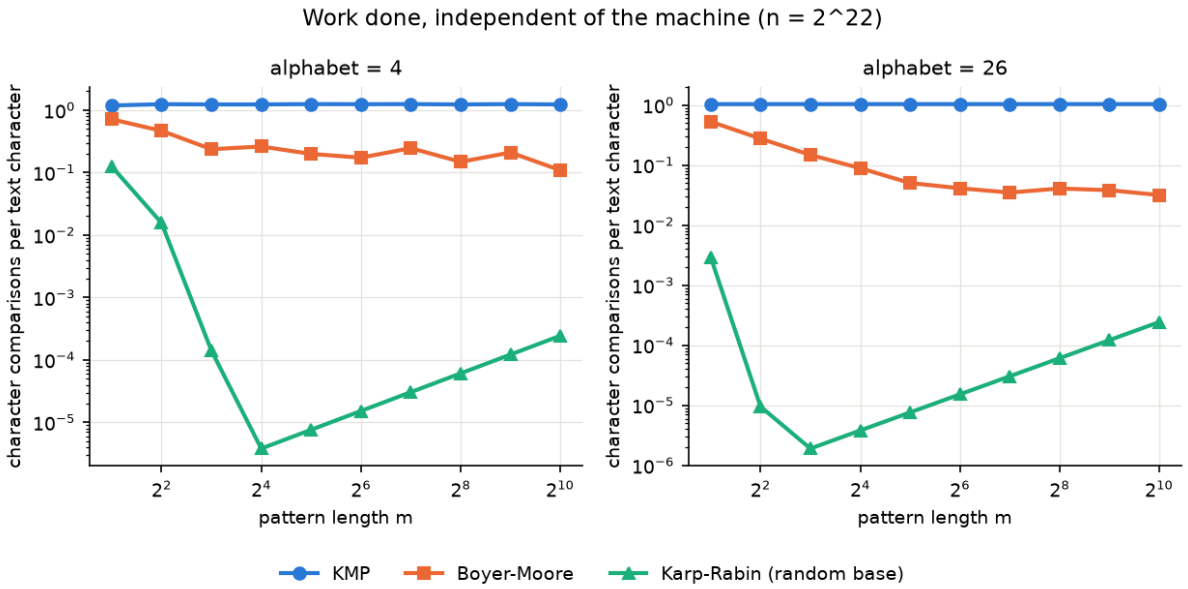


Figure 2. Comparisons per character against pattern length (same instance as Figure 1)

Figure 2 helps to explain why Boyer-Moore's advantage eventually stops growing. On alphabet of 26 letters, the time falls from 1.08 ns/char at m = 4 to 0.23 ns/char at m = 32 and then it barely moves (0.20 at m = 64 and 0.16 at m = 1024). The average shift, which is the text length divided by alignments, from comparisons.csv shows us the plateau. It is 20.7 at m = 32, 25.3 at m = 64 and 33.1 at m = 1024. Once the pattern is much longer than the alphabet, every letter will occur somewhere towards the right end, so the bad character rule can't shift by much more than σ.

On 4 letters, Boyer-Moore's time does not fall evenly (1.44 ns at m = 16, 1.35 at m = 128 and 0.62 at m = 1024). With such a small alphabet, the mismatching text character is likely to appear multiple times within the pattern, limiting the effectiveness of the bad-character rule. Performance therefore depends more heavily on the good-suffix rule, whose behaviour is determined by the pattern's internal repetition structure.

4.2 Alphabet size

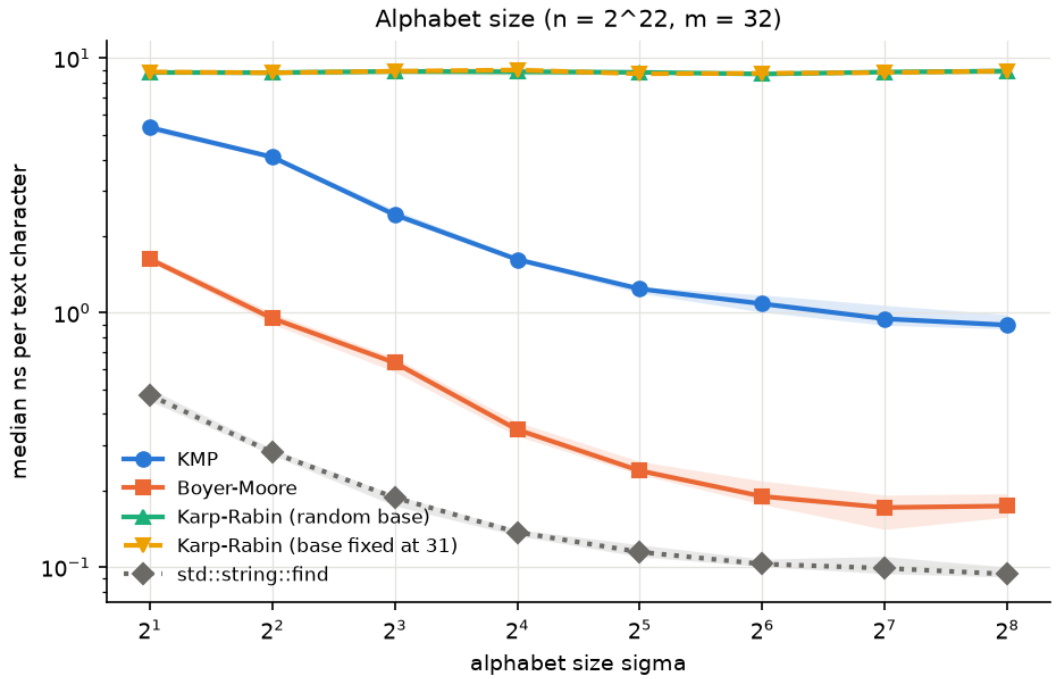


Figure 3. Time per text character against alphabet size ($n = 2^{22}$, $m = 32$)

Boyer-Moore matches our third prediction. From σ = 2 to σ = 256 its time falls 9.3× (1.62 to 0.17 ns per character) as its average shift grows from 7.3 to 30.1. KMP was also sensitive to the alphabet, as P3 said, and falls 6× (5.34 to 0.89 ns per character), even though its comparison count only falls from 1.40 to 1.00 per character. Karp–Rabin is flat at 8.7–8.9 ns, so P3 was wrong about Karp-Rabin.

Karp-Rabin barely noticed the text length, the pattern length or the alphabet. Its median stayed between 8.3 and 9.9 ns per character on every random input but one, from n = 16,384 to 16.8 million, pattern length from 2 to 1,024 and alphabet size from 2 to 256 letters, and the counted runs found no false positives. This is what the rolling hash predicts, because every window costs the same arithmetic. The one exception was m = 2 on four letters, where one window in 16 was a real match (262,800 in total) and each had to be checked, which pushed it to 10.7 ns. On 256 letters it was about 50 times slower than Boyer-Moore and 100 times slower than std::string::find, because both skip or scan past most of the text while Karp-Rabin hashes every window. The base 31 version matched the random-base version within 7% on every random input, so on ordinary text the choice of base costs nothing.

4.3 Text length

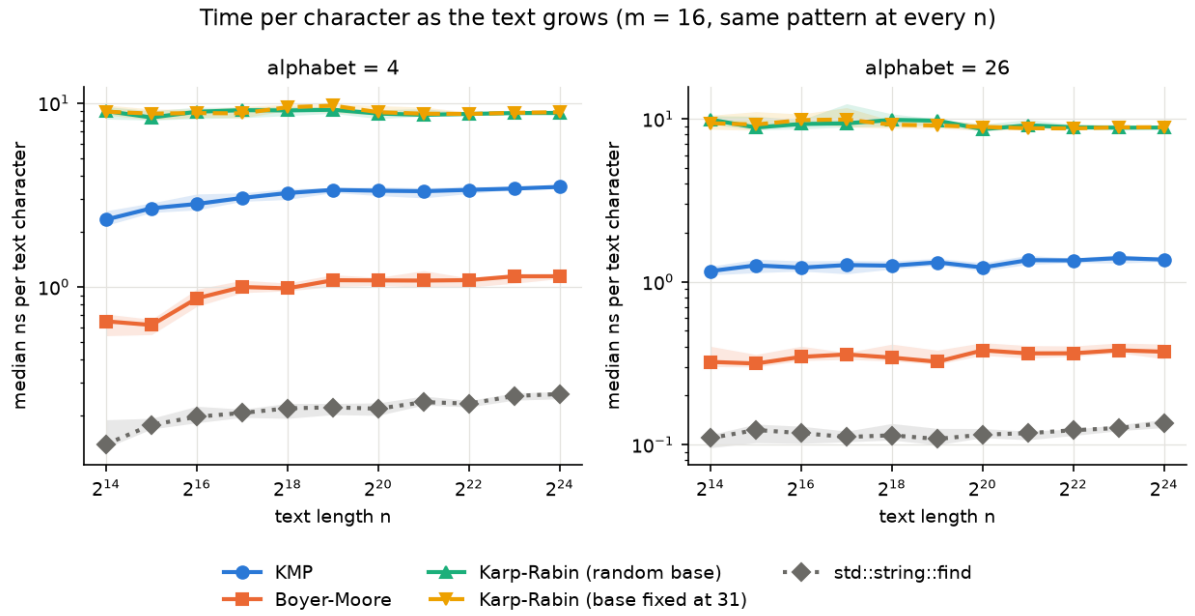


Figure 4. Time per character against text length (m = 16).
The operation counts grow in proportion to n, as P4 predicted: Boyer-Moore makes 0.21 comparisons per character at σ = 4 and 0.08 at σ = 26 at every length. At σ = 26 the time per character is also close to flat. At σ = 4 it rises with n, from 0.65 to 1.14 ns for Boyer-Moore and from 2.32 to 3.50 ns for KMP, even though the counts do not change. Section 5.4 explains why this is not a cache effect. In an earlier version of this experiment each n had a new random pattern. Boyer-Moore's comparisons per character then changed from one length to the next for reasons unrelated to n, so we fixed the pattern.

4.4 Repetitive and worst-case inputs

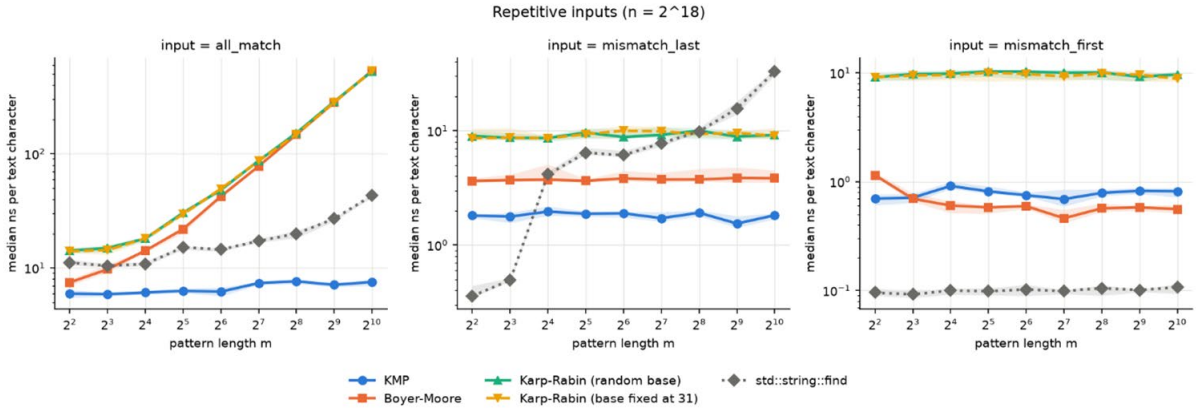


Figure 5. Repetitive inputs (n = 2^{18}): every position a match, mismatch at the last pattern character, and mismatch at the first.

Every position matches

This benchmark experiment is where the deterministic algorithms win. KMP stays between 5.9 and 7.6 ns per character at every m and makes exactly one comparison per character. Boyer-

Moore and Karp-Rabin both make exactly m comparisons per alignment, which is 4 per character at m = 4, and 1020 at m = 1024. Due to this, their time grows in proportion to m. Boyer-Moore is 71x slower than KMP at m = 1024. This confirms our 5th prediction and shows the O(nm) worst case of Boyer-Moore directly. KMP is slower here than on random text because it reports approximately 262,000 matches. KMP still avoids paying m comparisons per match, so this reduces the output cost.

Table 4. Text a^n, pattern $a^{m-1}b$, n = 2^{18}

Method	comparisons/char, m = 16	ns/char, m = 16	comparisons/char, m = 1024	ns/char, m = 1024
KMP	2.000	1.96	1.996	1.82
Boyer–Moore	1.000	3.74	0.996	3.84
Karp–Rabin (random base)	0.000	8.64	0.000	9.19
std::string::find	—	4.15	—	33.01

Mismatch at the last character.

Prediction 6 said that Boyer-Moore would win on this experiment, because it makes one comparison per character and KMP makes two. The counts in Table 4 match that, but the times do not. KMP took 1.8 to 2.0 ns per character and Boyer-Moore 3.6 to 3.8, so KMP was about twice as fast and prediction 6 was wrong. Boyer-Moore takes 3.6-3.9 ns per character at every m, which is about twice KMS's 1.5-2.0. This being half of the work and double the time means that each Boyer-Moore step costs about four times as much as a KMP comparison on this input.

Mismatch at the first character

For this experiment, both the deterministic algorithms perform well. Boyer-Moore compares the a's from the right, fails at the b, and the good-suffix rule then shifts it by m, so it averages one comparison per character, and takes 0.46-0.70 ns per character from m>= 8. KMP takes 0.69-0.91 ns. The ba^{m-1} patterns never occurs, so Karp-Rabin had no hash hits at all and ran at about 9 to 10 ns per character for every m, the same as on random text.

4.5 Karp-Rabin Random Base

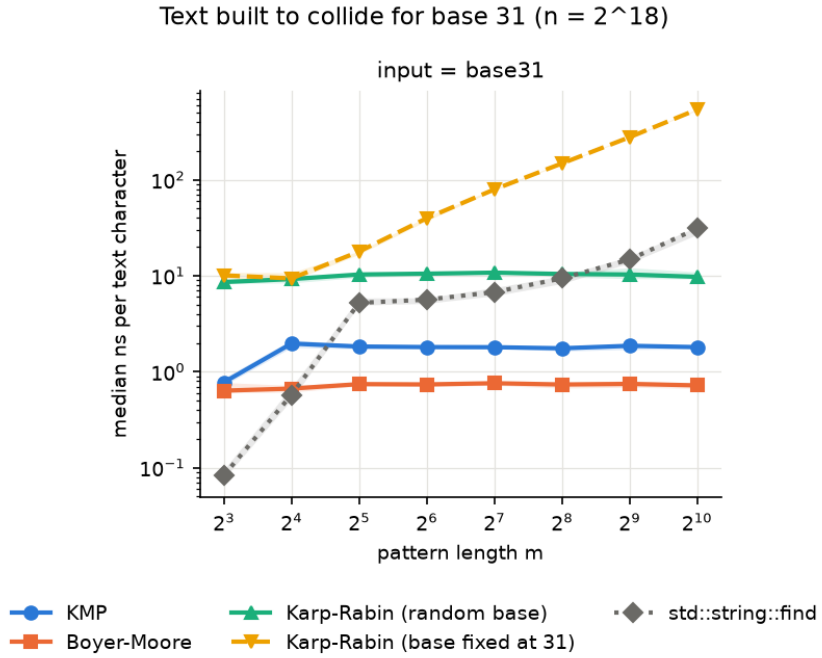


Figure 6. All five searches on text built to collide for base 31 ($n = 2^{18}$).

With the base fixed at 31, the built input turns every one of the n − m + 1 windows into a hash hit and a false positive: at m = 1,024 the counts show 261,121 false positives and 1,013 comparisons per character, and the time grows from 10.1 to 541 ns per character as m goes from 8 to 1,024. With a random base the same input produces no hash hits at all, and Karp-Rabin stays at 8.6-10.8 ns per character. So P7 was right, although the slowdown only starts at m = 32 (section 5.2).

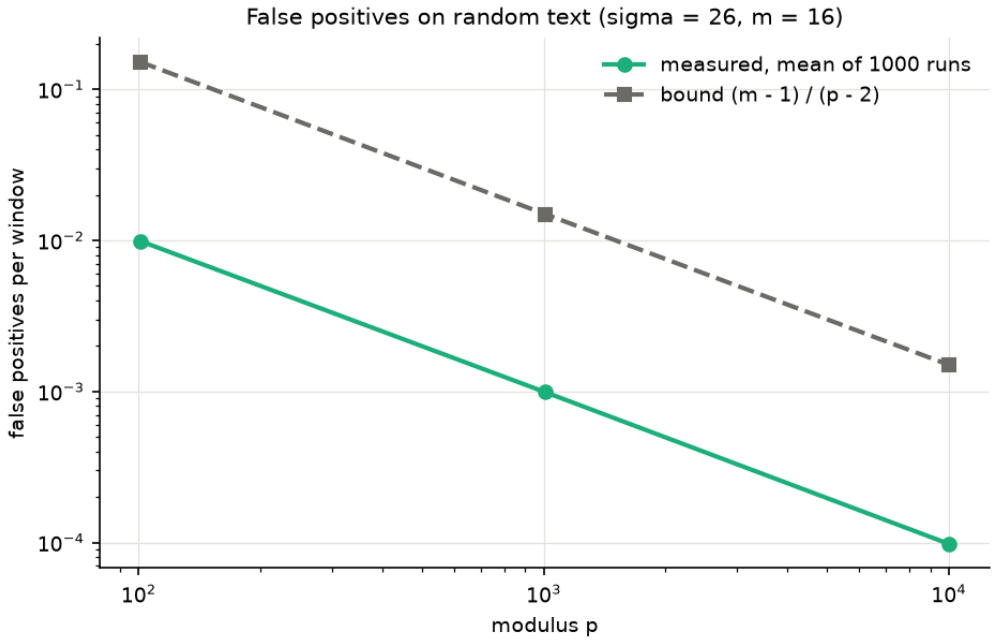


Figure 7. False positives per window on random text against the bound ($\sigma = 26$, m = 16, $n = 2^{16}$)

On random text the number of false positives followed (n − m + 1) / modulus very closely, with an average of 647.5 per search against the 648.7 we expected for a modulus of 101. For 1,009 we expected 64.9 and measured 64.8, and for 10,007 we expected 6.5 and measured 6.4. The theoretical bound of (m − 1) / (modulus − 2) per window is about 15 times higher than these rates, because the bound has to hold for every input and random text is far from the worst one.

4.6 Karp-Rabin Tail Behaviour

Table 5. 1,000 single runs per scenario (rk_tail_summary.csv)

Scenario	p	Method	median (ms)	99th pct (ms)	max / median	runs with a false positive	mean false positives
random text	$2^{31}-1$	Karp–Rabin	10.08	13.05	2.05	0	0
random text	—	KMP	3.76	5.27	2.48	—	—
random text	—	Boyer–Moore	1.35	2.12	3.95	—	—
random, small p	101	Karp–Rabin	0.630	0.913	1.66	1000	647.5
random, small p	1009	Karp–Rabin	0.613	0.855	1.57	1000	64.8
random, small p	10007	Karp–Rabin	0.587	0.946	1.82	999	6.4
adversarial	101	Karp–Rabin	0.587	10.01	20.48	48	3133.5
adversarial	1009	Karp–Rabin	0.595	0.941	1.76	0	0
adversarial	$2^{31}-1$	Karp–Rabin	0.596	0.884	1.85	0	0
adversarial	—	KMP	0.099	0.149	2.28	—	—
adversarial	—	Boyer–Moore	0.041	0.063	5.41	—	—

Every one of the 7,000 Karp-Rabin searches returned exactly the right matches, and the false positives were cheap: going from a modulus of 10,007 down to 101 added about 43 microseconds to a search of 0.587 ms, roughly 67 ns per extra false positive.

The adversarial input showed the heavy tail that comes with a random choice, because with a modulus of 101 exactly 4.04% of all possible bases make the adversarial pattern collide with a^m, a figure the benchmark found by trying every base. With one of those bases every window collides at once, so a search is either completely clean or completely bad. We saw 48 bad searches out of 1,000, close to the 40 we expected, and each one hit all 65,281 windows and took between 7.6 and 12.0 ms instead of the usual 0.59 ms, about 13 to 20 times slower. With a modulus of 1,009 only 0.2% of bases collide, so we expected about 2 bad searches and saw none, and with the real modulus there were none at all.

A deterministic algorithm always takes the same time on a given input, but Karp-Rabin's time depended on its base: almost always normal, and occasionally 20 times slower. With the real modulus the bound puts a bad base below one in eight million per window, which is why we never saw one.

5 What surprised us

5.1 Binary text: Boyer-Moore beat KMP

Prediction 3 said small alphabets would slow Boyer-Moore down. On binary text ($\sigma = 2$, $m = 32$) it was still 3.3 times faster than KMP: 1.62 ns per character against 5.34.

On Boyer-Moore's side, the good suffix rule did the work. With two letters, both appear near the end of the pattern, so the bad character rule can only move it a place or two. Its average shift was still 7.3. In a random binary pattern, a matched run of several letters often does not appear again with a different letter in front, so the good suffix rule can move the pattern a long way.

On KMP's side, its comparisons only rose from 1.00 to 1.40 per character between $\sigma = 256$ and $\sigma = 2$, but its time rose 6 times. A DFA version with no fallback loop ran at 1.57 ns on binary text, about the same as on 256 letters, while KMP took 5.28. So the extra time comes from the fallback loop, which runs 0.40 times per character on binary text. Our explanation is that the processor cannot predict when a fallback will happen.

5.2 Karp-Rabin collisions were free for short patterns

Prediction 7 said base 31 would slow Karp-Rabin to $\Theta(nm)$ on the adversarial input. It did for long patterns, but for short patterns the collisions cost almost nothing. At $m = 16$ every one of the 262,129 windows was a false positive needing 9 comparisons, yet base 31 took 9.38 ns per character against 9.23 for the random base, which had no hash hits.

The cost only appeared from $m = 32$, where each check needed 25 comparisons and base 31 took 17.87 ns against 10.30. At $m = 1{,}024$ it reached 540.7 ns, within 2.5% of the 527.7 ns Karp-Rabin took on a^m, where every window is a real match.

Our explanation is that each hash update has to wait for the one before it, so the processor spends most of each step waiting for multiplications and divisions. The comparisons in a check do not depend on the hash, so they can run in those gaps. Nine comparisons fit in the gaps, but 25 or more do not.

5.3 KMP slowed down on small alphabets

We expected KMP's time to be flat in σ (P3), because its comparison count only varies between n and 2n. Instead it was 6× slower on a 2-letter alphabet than on 256 letters. The counts of this leads to an analysis of the fallback look. Fallbacks per text character fall roughly as 1/σ (0.40 at σ = 2, 0.125 at σ = 8, 0.004 at σ = 256), and KMP's time of running also starts to decrease. We predict this may be related to predictions made. If a fallback happens at any given character, it will depend on the text which is random. Due to this, the processor cannot guess this character, and every wrong guess is then a misuse or throwaway or work already completed. On a large alphabet, predict that with a mismatch, k stays 0.

σ	KMP (ns/char)	KMP as a DFA (ns/char)	Karp–Rabin (ns/char)	KMP fallbacks/char
2	5.28	1.57	8.91	0.404
4	4.80	1.52	8.71	0.250
8	2.77	1.62	9.00	0.125
16	1.69	1.68	8.93	0.062
32	1.24	1.59	8.85	0.031
64	0.98	1.59	9.05	0.016
128	0.90	1.55	9.17	0.008
256	0.86	1.63	9.08	0.004

Table 6. KMP, KMP as a DFA and Karp-Rabin on the alphabet inputs (n = 2^{22}, m = 32, explain_kmp_dfa.csv)

We wanted to test the above explanation of the branch prediction. To do so, we removed the unpredictable branch, which should then remove the dependence on the alphabet size. This version prepopulates KMP's transitions as a DFA, so that the search is one table lookup per character with no inner loop state = dfa[state][c]. This runs on the same instances. When this was run, the predictions held up. The DFA's time stays between 1.52 and 1.68 ns per character at every alphabet size, while KMP's falls from 5.28 to 0.86. At σ = 2 the DFA is 3.4× faster than KMP. The two cross at σ = 16 (1.69 against 1.68), and above that KMP is faster. We think this is because the DFA's lookup reads a table of (m + 1) × 256 entries in which each load depends on the previous one. The asymptotic complexity remains the same for both, O(n).

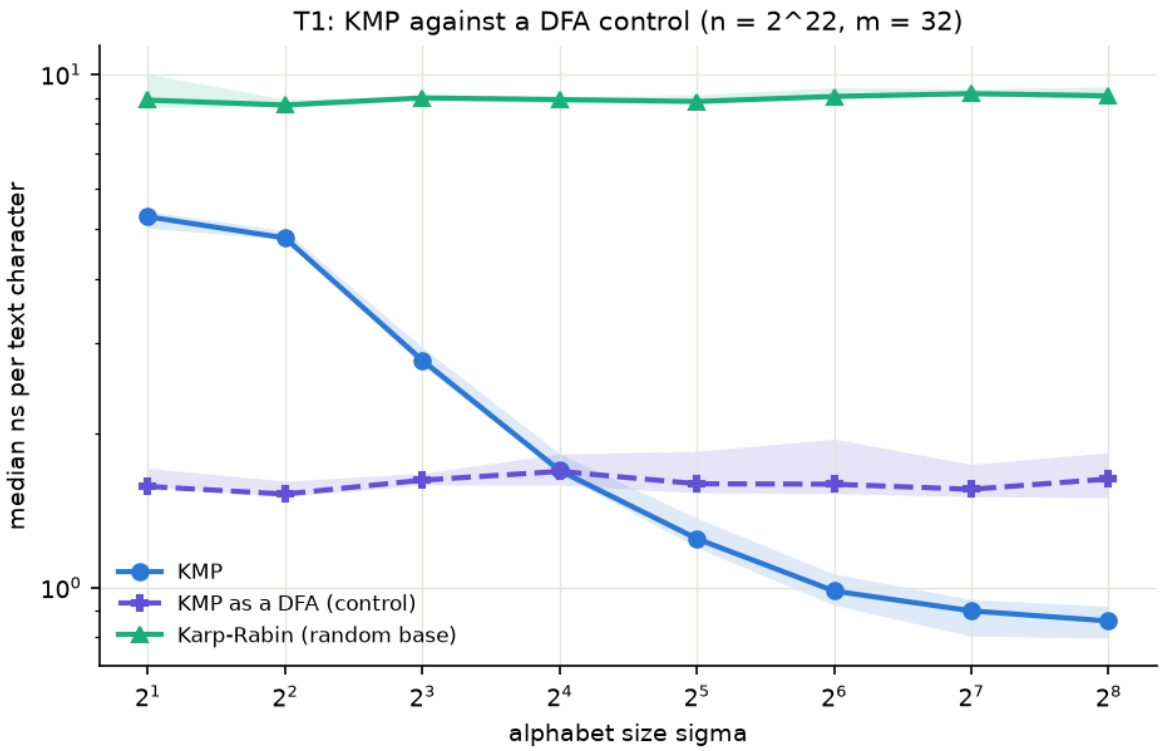


Figure 8. KMP against a DFA version of KMP on the same instances as Figure 3. Karp Rabin is shown for reference.

5.4 Longer text costs more per character on small alphabet size

scale_n uses the same pattern at every n (Table 2), so the work per character is identical at every text length. We expected linear and flat lines (P4). At σ = 26 they nearly are; at σ = 4 KMP's time per character rises by 51% (2.32 to 3.50 ns) and Boyer–Moore's by 76% (0.65 to 1.14 ns).

A 16 KB text fits in the processor's fastest cache and a 16 MB text does not. KMP reads every byte of the text in order at both alphabet sizes, so it moves the same data through the cache whether σ is 4 or 26; so a cache effect should add about the same time per character to both. Instead KMP's rise is about 1.2 ns per character at σ = 4 and about 0.2 ns at σ = 26.

Our best explanation is a measurement effect of repetition. For short texts, each timed sample repeats the search on the same text many times (to reach 5 ms), and modern branch predictors use long histories, so they may partly learn the sequence of branch outcomes for a text they have seen many times. Longer texts exceed what the predictor can remember, and the mispredictions return. This would raise the time with n only where branches are unpredictable, which is what we see.

6 What we'd do next

Testing our explanations
Hardware counters would show whether KMP's fallbacks cause branch misses (5.3) and whether Karp-Rabin's checks run while it waits on the hash (5.2). For 5.2, a cheaper hash should make the slowdown start at a smaller m. For the rise with n in 5.4, rerunning scale_n with the KMP DFA, or with a different text in each repeat, should remove the rise with n if our explanation is right.

std::string::find on 256 letters
We only tested m = 32 there, where find was 1.8 times faster than Boyer-Moore. Boyer-Moore's shift keeps growing with m, so it might overtake find on longer patterns.

A faster hash with the same guarantee
Karp-Rabin's 9 ns per character is almost all arithmetic. We would like to try the prime $2^{61} - 1$, whose remainders can be found with shifts and additions instead of division, and see how close Karp-Rabin can get to KMP while its false positive bound becomes even smaller.

7 Individual contributions

Aeesha Bello

I worked on what is shown in the repository as the final benchmarking framework. I ran the scripts and the benchmarking and worked on the analysis and verifying the results and graphs. I implemented the Cross tests which tests all the benchmarked algorithms against the naïve reference. I also implemented the explanation framework and its tests which was used for the KMP DFA explanation on changing one thing to see the results on small alphabet sizes. In the report, I contributed heavily to the Results and Analysis in Section 4, I completed the analysis of the results and the graphs. I also helped write the predictions and the breaking down the experimental methodology and how to test correctly.

Rebecca Kim

I wrote the first version of the benchmarking program, which the final framework was built from. I reviewed other members' code and tests and reported the problems I found and fixed them. When the final repository was put together, I fixed the tests and build files so that reproduce.py runs every step, and helped with the README. I checked the numbers and results in the report against our results files. In the report, I worked on group sections, including the theoretical comparison, the experimental methodology, the results and analysis, and the surprises. I also edited the whole report to remove repetition and fix contradictions between sections.

Mallory Li

I used Claude, for the investigation and supplemental help in debugging for my implementation of Karp-Rabin. This included testing implementation against a naive search on 200,000 small random inputs, using tiny moduli so false positives happened constantly. I also used it to

quality control the Karp-Rabin parts of the scripting process and reporting to make sure that everything was consistent at different points of our joint implementation.

Claude was wrong or out of date in a few places, so I didn't take its output on trust. At one point of the report used numbers from an older version of our benchmark. When our group committed the official results, two of its claims no longer held: that Karp-Rabin beat Boyer-Moore on all a's, and the pattern length where fixed-base collisions start to cost time. Both were rewritten from the committed CSV files. Every Karp-Rabin number in the report now comes from those files, and the counts can be checked by hand against the n − m + 1 and (n − m + 1) / modulus rules.

8 AI use

8.1 How each member used AI

8.1.1 Aeesha Bello

I used Copilot to explain how to organise the repository effectively. This gave me information with the set up and build system and how to separate the benchmarking from the algorithms. This was used exactly to scaffold the repository. I used Claude to expand on our entire existing benchmarking framework, to ensure that all the inputs and experiments were valid and constructive to show results for analysis. This fleshed out our input generators, how to run the benchmarking, what the key results should look like for analysis and utilising matplotlib. The most helpful part of this was in updating our first experiment **scaleN** to test the same pattern for every n using different alphabet sizes. I used the code for this on the runner, main, and explanation tests. I also used Claude entirely to write both scripts to run the experiments and make the plots.

8.1.2 Mallory Li

I used Claude, for the investigation and supplemental help in debugging for my implementation of Karp-Rabin. This included testing implementation against a naive search on 200,000 small random inputs, using tiny moduli so false positives happened constantly. I also used it to quality control the Karp-Rabin parts of the report to make sure that everything was consistent at different points of our joint implementation. Further, in the scripting process for the video I used Claude to pace my sections and keep myself in time, as well as education myself on using Canva more effectively as I had limited experience in using it before.

8.1.3 Rebecca Kim

I used Claude mostly to draft first versions of my work and to explain things I did not understand. It explained how Boyer-Moore's good suffix table is built from the pattern's borders, how the parts of the assignment fitted together, and why some of our results behaved the way they did. It drafted the first version of my Boyer-Moore tests by suggesting test cases, and the first version of the group's benchmarking program, which the group later replaced with a cleaned-up framework. I also used it to double check the other members' code by running it against a naive search. When the final repository would not build, I used it to find what needed fixing. For the report, it drafted wording I was struggling with, checked our numbers against the results files, and helped me cut repeated content to meet the word limit. I also used it to draft

my video slides, the script for the results and surprise parts, and an outline for my reflection, which I then edited and rewrote in my own words. I checked everything it gave me against our own code and results before using it.

8.2 Where AI was wrong, unhelpful, or misleading

- It gave an explanation that our own data ruled out. At σ = 4, KMP's time per character rose from 2.32 to 3.50 ns as the text grew, even though its comparisons per character did not change. The AI said this was because the larger texts no longer fit in the cache. It sounded right, so we nearly used it. Then we checked σ = 26 in runtime_summary.csv. KMP reads every byte in order at both alphabet sizes, so a cache effect should add about the same time to both. But KMP rose about 1.2 ns at σ = 4 and only 0.2 ns at σ = 26 (1.16 to 1.37). We dropped the cache explanation, wrote the result up as a surprise and listed a test for our own explanation in section 6.
- We used AI for the first draft of our first experiment (the varying text length) and this code generated a new random pattern for every text length. The code for this looked correct and the plots produced were reasonable. This, however, was incorrect and unhelpful as the operation counts showed Boyer-Moore's comparisons per character jumping between (0.21 to 0.35) for reasons which were not obvious to us and were unrelated to the text length. Only the pattern was changing, and not the length of text **n**. We rewrote this experiment so that each alphabet uses one long random text, and every n is a prefix of it. The same pattern is used at every n. The comparisons per character for these then became constant and we mention this original mistake in section 4.
- We used AI when investigating an alternative implementation for the KMP as a DFA idea. The AI correctly explained the theory of a DFA for string matching and how that can be described in terms of pattern length and alphabet size. Originally, the AI said its table has m x sigma entries (pattern length x alphabet size However, this was not directly applicable to our implementation. Our DFA represented transitions for all possible byte values, so the table needed to contain (m + 1) × 256 entries regardless of how many characters happened to appear in the input text or pattern. Using the AI's suggested structure produced an incorrectly sized transition table, which caused search failures on test cases containing characters that were not explicitly represented in the reduced alphabet. We caught this through testing because the implementation failed tests and by debugging, we discovered that the transitions for some character values were never created.
- It described benchmark steps our code does not do. Its draft of helping word a few draft sections said reproduce.py pins the benchmark to one core with taskset, and that results/summary/winners.csv marks which differences are clear of noise. Neither is in the final repository. Reproduce.py has no taskset, and plot_results.py writes no winners.csv, The draft has been written against an earlier version of the code. Anyone rerunning our experiments from section 3 would have looked for steps that did not exist. We checked the files and settings named in section 3 against the final code and kept only what is there.

8.3 AI in group work

We used Claude to explain both Boyer-Moore and Karp-Rabin algorithms to ensure the analysis of results and observations were connected to the structure of the algorithms. This gave us depth to the analysis as we could point out certain sections and why certain results showed things. When we first used AI for analysis, it gave numbers that did not match our data. It wrote that KMP took 1.8 to 2.0 ns per character on $a^{m-1}b$ in a^n, but it had only looked at some of the pattern lengths. We checked every row of runtime_summary.csv, found KMP went as low as 1.54 at m = 512, and used the full range of 1.5 to 2.0. Overall, the AI was extremely effective in organising the benchmarking framework and the foundational functionality to make sure all implementations would work effectively and return the correct data needed for analysis.

9 References

Boyer, R. S., & Moore, J. S. (1977). A fast string searching algorithm. *Communications of the ACM, 20*(10), 762–772. https://doi.org/10.1145/359842.359859

ByteQuest. (2024, January 21). *Knuth-morris-pratt algorithm visually explained*. YouTube. https://www.youtube.com/watch?v=ynv7bbcSLKE

GeeksforGeeks. (2025, April 12). *Boyer Moore algorithm | Good suffix heuristic*. https://www.geeksforgeeks.org/boyer-moore-algorithm-good-suffix-heuristic/

GeeksforGeeks. (2011, April 3). *KMP algorithm for pattern searching*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/

Prefix function - knuth-morris-pratt - algorithms for competitive programming. (2023). Cp-Algorithms.Com. https://cp-algorithms.com/string/prefix-function.html

Yoshi. (2025, January 4). *[For beginners] understanding the KMP algorithm by comparing with the brute-force*. DEV Community. https://dev.to/yo-shi/for-beginners-understanding-the-kmp-algorithm-by-comparing-with-the-brute-force-1da3

10 Appendix A: Individual reflections

10.1 Karp-Rabin – Mallory Li

What's the worst case for Karp-Rabin, and what kind of input triggers it?

The first worst case is text with a real match at almost every position, such as a^1024 searched in 2^18 a's, and it is the one that randomness cannot fix. Every one of the 261,121 windows has the right hash because it really is a match, so each one has to be checked in full. That came to 267,387,904 comparisons, about 1,020 per text character, and the search ran at 527.7 ns per character against about 9 on random text. KMP did the same search at 7.5 ns. So, no base can help here, because these hits are not collisions.

What was the hardest bug or design decision, and how did I resolve it?

The hardest decision was whether the first character of a window should get the highest power of the base or the lowest, since both give a correct search. I kept the textbook order with the first character highest, and this decision did end up mattering because the adversarial generator rebuilds my hash in polynomialHash and relies on the last eight characters of the pattern carrying the lowest powers. With the order reversed, a tail that collides at the end of the pattern would no longer make the whole window collide. The cross test that checks every window is a false positive under base 31 confirmed that the two hashes agree.

If I did this implementation again from scratch, what would I do differently?

I'd think more about the cost of the arithmetic from the start, because the modulus is passed in when the program runs and so every update needs real division instructions. A modulus fixed at compile time, or the prime 2^61 − 1 with its division free shortcut, might bring the 9 ns per character down a long way while keeping the same guarantee.

I'd also build the adversarial input alongside the algorithm rather than after it, since seeing the fixed base version pushed to 540 ns per character taught me more about why the base is random than the proof did.

What's one thing about Karp-Rabin I didn't fully understand before starting, and now do?

I thought the random base was there to make collisions rare on average, like how random text makes them rare, but it is actually a much stronger promise than that since the probability is taken over the base and not over the text. So the bound holds for every input, even one built by someone who wants it to fail, as long as they cannot see the base. The adversarial input showed this clearly, since with base 31 every window collided and with a random base the same input gave no hash hits at all.

10.2 KMP – Aeesha Bello

KMP's worst case occurs when the pattern contains long repeated prefixes and suffixes, causing frequent fallback operations through the LPS table. Inputs such as a text consisting entirely of 'A' characters and a pattern of the form 'AAAAAB' maximise the number of fallbacks. Despite this, KMP remains linear because each text character participates in at most a constant number of comparisons and the text pointer never moves backwards.

The hardest part of implementing my algorithm was understanding how to construct my Longest Prefix Suffix (LPS) table. Initially, I though of the LPS array as storing repeated characters only, but it's actually used to record how much of a previous match that can be reused after a mismatch. The hardest design decision was to handle any mismatches during the construction of the LPS and searching. When a mismatch happens, KMP uses values already stored in the LPS table to determine the next valid pattern position. I needed to understand why both

length = lps[length - 1] and k = lps[k - 1]

work by going through examples by hand and manually seeing how the prefixes and suffixes overlap. I worked through a specific pattern **AAAAAB** character by character and manually constructing the LPS table. After matching several A characters, a mismatch means the work is still useful. Because the pattern contains repeated prefixes, KMP can immediately continue from the longest valid prefix instead of restarting from the beginning. This gave me more depth in understanding of prefixes and suffixes and then the rest of the algorithm was easier to understand and debug.

I would add more instrumentation into the implementation from the beginning. I added the comparison and fallback counts later in my implementation which was very helpful in the benchmarking but also for understanding if the algorithm was working and expected. Having these counters available earlier would have made debugging easier. When I added the counting functionality, and started testing the text **AAAAAAAAAA**, with the pattern **AAAAAB**. The stats for this showed a high number of fallback operations. This confirmed that the algorithm was using the LPS table correctly. I would potentially implement the counting functionality into the implementation from the beginning because it shows clarity on what the algorithm is doing internally.

The hardest thing for me to understand at the beginning was why the text pointer (i) never moves backward during a mismatch. In a normal naïve or brute-force search, In a brute-force search, when you match 5 characters of a pattern and then the 6th character fails, you naturally slide the pattern over by one step and restart matching from the beginning of the pattern or step the text pointer back. KMP does not re-scan characters, the pattern pointer (j) moves backward to the calculated index using *lps[j-1]*. This also provided an explanation for why the worst-case complexity is **O(n+m).** The efficiency of the algorithm comes from the text index not moving backwards and making sure that every mismatch uses information already learned.

10.3 Boyer-Moore – Rebecca Kim

What's the worst case for Boyer-Moore, and what kind of input triggers it?

Text of all a's, searched for a pattern of all a's. Every position is a match, and after each match the pattern can only move by goodSuffix[0], the pattern's period, which is 1 here. So each alignment checks all m characters again: (n – m + 1) × m comparisons. For n = 262,144 and m = 256 that's 67,043,584, about 256 per text character against KMP's 1. It ran about 19 times slower than KMP.

Patterns that repeat with a longer period do this too. For abab…ab searched in abab…, the pattern moves 2 places after each match, so it does about m / 2 comparisons per character.

What was the hardest bug or design decision, and how did I resolve it?

The second pass of the good suffix table, which handles suffixes that don't appear again in the pattern. My first version filled those entries with m, the full pattern length. For abcab in abcabcab there are matches at 0 and 3, and a shift of 5 jumps over the second one.

I fixed it by using the borders: start from the longest border of the whole pattern, and once you pass it, drop to the next shorter one. For abcab that gives 3 3 3 3 5 1.

To check it, I wrote goodSuffixByDefinition, a slow version that tries every shift straight from the rule's definition. tests/boyer_moore_tests.cpp checks the real table against it on 3,000 random patterns over {a,b,c}. When I set those entries to m again, 12 of the 29 tests failed.

If I did this implementation again from scratch, what would I do differently?

I'd write goodSuffixByDefinition first and test against it from the start, instead of adding it after the O(m) version.
I'd use std::ptrdiff_t for indexes instead of int, to remove the 2^31 character limit noted in src/boyer_moore.cpp.
I'd also build counting into the search from the start. The counted version is a separate copy of the loop, so a bug could be in one copy and not the other.

What's one thing about Boyer-Moore I didn't fully understand before starting, and now do?

I thought the good suffix rule was a small extra on top of the bad character rule. On binary text with m = 32, both rules together move the pattern 7.3 places on average (comparisons.csv). With only 2 letters, both usually appear near the end of the pattern, so the bad character rule alone usually can only move it a place or two. The good suffix rule uses the whole matched run, which in a random binary pattern often doesn't appear again with a different letter in front, so it can jump much further. This helps explain why Boyer-Moore was 3.3 times faster than KMP on binary text, even though P3 said a small alphabet would slow it down.